Faculty of Science and Engineering
Department of Advanced Computing Sciences

Open source eye-tracking for accessible communication: Project Plan

MSc DSDM & MSc AI

Research Project - Group 10

Authors

Mathijs (i6320858)
Dan (i6320800)
Gabriel (i6321439)
Vasile (i6321735)

Supervisors

Dr. Rachel Cavill
Dr. Gaurvi Goyal

Coordinator

Dr. Linda Rieswijk

Contents

1 Context and Motivation **1**

2 State of the Art **2**

3 Concepts and Approach **2**
3.1 Gaze Tracking . . . 2
3.2 Head Movement and Pose Compensation . . . 3
3.3 The Communication Frontend: Cboard . . . 3
3.4 Application Architecture . . . 4
3.4.1 The Flutter Frontend . . . 4
3.4.2 The Python Backend (The Inference Engine)(maybe not necessary) . . . 4
3.4.3 Architectural Advantages . . . 4
3.5 Evaluation and Testing . . . 5
3.5.1 Algorithmic Accuracy and Precision . . . 5
3.5.2 Robustness to Head Movement and Environmental Variables . . . 5
3.5.3 System Latency and Architectural Performance . . . 5

4 Deliverables **5**

5 Time Management **6**

6 Risk analysis **7**
6.1 Gaze Tracking Risks . . . 7
6.2 System Latency and Architectural Bottlenecks . . . 7
6.3 Project Scope and Implementation Complexity . . . 7

7 Overview of who did what **8**

Bibliography **8**

1 Context and Motivation

The domain of Augmentative and Alternative Communication (AAC) represents a critical intersection of human-computer interaction, assistive technology, and clinical rehabilitation. For many children and adults living with severe speech and physical impairments, AAC systems are not merely convenience tools; they are fundamental instruments for maintaining autonomy [1]. Many disabled individuals traditionally rely on communication books or equivalent applications where they select from a range of images to communicate. However, when physical disabilities prevent the use of traditional touch interfaces, oculomotor function often remains the only channel for intentional interaction, making eye-tracking systems the cornerstone of advanced AAC interventions.

A profound disparity currently exists between the capabilities of state-of-the-art commercial systems and their real-world availability. Typical medical-grade, commercially available AAC eye-tracking systems provide robust and essential services for their non-verbal users. Yet, they rely heavily on proprietary hardware, specialized processing units, and active infrared cameras. The high costs required to operate these high-tech AAC devices could hinder the access [2]. This results in inaccessibility to a vast majority of potential users, particularly people in low-income families or under-resourced healthcare systems.

The primary motivation of this project is to democratize access to this essential technology and help give a voice to children and adults in need. This project will investigate what can be achieved by using and extending open-source libraries and the standard camera on consumer devices, such as an Android tablet or iPad. These devices do not have access to specialized infra red cameras and usually possess low-power processing units. Mobile devices are far more widespread than the number of desktop/laptop users [3]. By coupling software-driven gaze estimation algorithms with free, open-source AAC platforms, we seek to eliminate the dependency on expensive, specialized hardware and create a viable, low-cost alternative.

The aim is to build a functional system that can accurately detect which of the images presented on the screen the user is looking at. The final system would be made open-source and freely available to any users in the future. Crucially, it must be easy for users and caregivers to set up, assess for ease-of-use, and extend with new image sets, ensuring that advanced communication tools are finally within reach for those who need them most.

The research questions we intend to answer are:

- How effectively do existing gaze-tracking libraries perform for our intended use?
- Can head tracking compensate for head movement during gaze tracking?
- What impact does wearing glasses have on gaze-tracking performance?
- How accurate are these methods when implemented on standard consumer tablets?

2 State of the Art

The current state-of-the-art (SOTA) in eye-tracking for assistive technology is deeply bifurcated. On one end, commercial systems represent the gold standard, offering high precision through specialized infrared hardware and proprietary software [4]. On the other end, the open-source community has developed incredibly powerful, lightweight computer vision libraries capable of running on standard RGB cameras [3], such as MediaPipe [5], EyeGestures [6], and OpenFace. However, a fully integrated, software-only AAC system that bridges these technologies for standard consumer tablets does not yet exist. While the underlying SOTA algorithms are freely available, they remain fragmented toolkits. Utilizing them requires substantial software engineering expertise to architect data pipelines, map complex 3D geometry to 2D screen coordinates, and build responsive user interfaces. Consequently, people with non-technical backgrounds such as parents, caregivers, and speech-language pathologists, cannot realistically develop or configure such a system for themselves [2]. Packaging these advanced, disparate SOTA components into a cohesive, out-of-the-box application is therefore an urgent necessity to translate theoretical technological capabilities into practical, everyday accessibility [1, 2].

3 Concepts and Approach

To realize an accessible AAC system, this project adopts a decoupled, software-centric approach to replace expensive proprietary hardware. The foundation relies on video-based gaze estimation using standard RGB cameras. However, to ensure continuous usability in real-world environments, the system must also integrate robust head-pose compensation to account for natural physical movement.

These computational processes are structured within a split architecture: a Python-based inference engine handles complex computer vision tasks in the background, continuously feeding real-time coordinate data to a responsive frontend. This frontend translates gaze data into actionable selections and text-to-speech output. Finally, a rigorous evaluation methodology will assess the system's accuracy, precision, and resilience to free-head movement and corrective eyewear.

3.1 Gaze Tracking

For the proposed communication system to function, its core requirement should be reliable eye tracking. The interaction with the system should be gaze based. That means that a user selects a word simply by looking at the corresponding panel. Therefore, the system must be able to estimate gaze direction accurately enough to map the user's attention to a specific on-screen item.

A key constraint of this project is accessibility. Many commercial eye-tracking systems rely on infrared illumination and corneal reflection techniques, in which infrared light sources illuminate the eye and both pupil centers and corneal reflections are detected using specialized cameras. [4] While these methods offer high accuracy, they require dedicated hardware and require higher costs and computational overhead. This project aims at enabling eye-tracking-based communication on standard consumer devices such as laptops or tablets.

Therefore, this project focuses on video-based eye tracking methods that operate using standard RGB cameras and have relatively low computational demands. Several open-source solutions meet

these criteria and will be investigated. The first option is EyeGestures [6], an open-source library designed for eye tracking using native webcams. EyeGestures estimates gaze direction by detecting facial landmarks and modeling eye movement without the need for infrared hardware. This makes it suitable for low-cost and widely available devices.

A second option is EyeTrax [7], another open source eye-tracking framework that operates with standard cameras. EyeTrax supports facial feature extraction, model training, and gaze prediction, and allows for extensive fine-tuning.

Finally, Owleye [8] will also be evaluated. This solution similarly relies on conventional webcams and offers a comparatively simple pipeline for gaze estimation.

3.2 Head Movement and Pose Compensation

Traditional appearance-based 2D webcam eye trackers are extremely sensitive to natural human head movements. The human head operates with six degrees of freedom (6DoF), meaning it can translate along the X, Y, and Z axes and rotate across pitch, yaw, and roll angles. In a purely 2D tracking system, shifting posture alters the pupil's pixel coordinates within the image frame, causing the software to register a false gaze trajectory even if the user's actual point of regard is perfectly stationary. Since AAC users cannot be expected to maintain a completely stationary head posture for extended periods, fixed-head gaze estimation fails to meet usability requirements [9].

To resolve this, the system must implement head-pose invariant gaze estimation. This requires the computational pipeline to continuously monitor the orientation of the head and mathematically separate it from the rotation of the eyeball. To execute this efficiently on standard consumer devices, the proposed system will implement Google's MediaPipe framework [5].

MediaPipe utilizes a highly optimized, two-step detector-tracker pipeline designed for mobile processors. Initially, a BlazeFace detector isolates the facial region of interest (ROI) across the full image. Subsequently, a Face Mesh model infers a highly dense topographical map consisting of 468 3D facial landmarks in real-time [10]. To maintain the high frame rates necessary for fluid communication, MediaPipe minimizes computational latency by using the landmark coordinates from the previous frame to crop the subsequent input, thereby circumventing the need to continuously re-detect the face.

By matching a pre-defined 3D canonical face model against the rigid 2D landmarks extracted by MediaPipe (such as the nose tip and orbital cavities), the system continuously computes the rotational matrices and translational vectors of the user's head using a Perspective-n-Point (PnP) algorithm [11]. Furthermore, MediaPipe encompasses an Iris Tracking module that isolates the eye contours and pupil. Because the human iris maintains a consistent horizontal diameter of approximately 11.7 millimeters, the framework can utilize this biological constant to perform depth-from-iris calculations [10]. This mathematically determines the exact metric distance from the camera to the user without requiring specialized depth sensors.

When the head posture deviates beyond a predefined tolerance, a dynamic reference update recalculates the expected "head-pointing" coordinate. The system utilizes the newly tracked spatial position of the eye corners as a stable anchor. A 3D-to-2D geometric projection then computes the difference vector between the current pupil center and the dynamic anchor [4]. Intermediate spatial relationships required for this 3D vector calculation are defined as:

$$O'_e C_1^2 = O'_e C^2 + \frac{C_2 C_1^2}{4}$$

By multiplying this difference vector by scale coefficients established during an initial calibration phase, the algorithm accurately outputs the exact 2D pixel coordinate where the user's gaze intersects the screen [4].

3.3 The Communication Frontend: Cboard

Cboard is a free, open-source AAC web application designed to assist children and adults with speech and language impairments [12]. In this project's architecture, Cboard acts as the primary user interface and interaction layer, replacing the need for proprietary and expensive communication software.

Built upon the React JavaScript framework, Cboard utilizes a component-based architecture to render highly customizable communication grids. A foundational feature of the application is

its integration with modern web standards, specifically the Web Speech API [12]. By utilizing the `SpeechSynthesis` interface, Cboard natively generates text-to-speech (TTS) output whenever a user selects a symbol or pictogram. Additionally, the platform leverages the ECMAScript Internationalization API (via the `react-intl` library) to provide comprehensive support for over 40 languages, ensuring global accessibility [12].

To address environments with unreliable internet connectivity, Cboard is engineered as a Progressive Web App (PWA) [12]. By implementing Service Worker APIs, the application can cache its core assets locally, including its default library of over 3,400 Mulberry pictograms. This allows the communication board to maintain full offline functionality after the initial browser load [12].

Within the context of our decoupled system architecture, Cboard will be modified to accept external gaze input. A custom React component will be developed to act as a WebSocket client, ingesting real-time coordinate payloads broadcasted by the Python-based computer vision backend. The React frontend will utilize this continuous data stream to render a dynamic visual cursor overlay and programmatically manage the dwell-time selection logic, seamlessly translating the user's point of regard into actionable AAC speech output.

3.4 Application Architecture

To build a functional communication tool, the project plans to utilize a decoupled or split architecture. This design divides the system into two primary components: a frontend, which manages the user interface and visual elements, and a backend, which handles the computational logic. By separating these concerns, the system can maintain a simple and responsive interface while allowing complex mathematical calculations to run independently in the background.

3.4.1 The Flutter Frontend

The user interface will be developed using the Flutter framework [13], chosen for its ability to provide a clean, modern design that can be deployed across various consumer devices such as tablets and laptops. In this proposed setup, the Flutter application is responsible for three core functions. First, it displays the communication grid, which contains the words or images available for selection. Second, it provides real-time visual feedback by rendering a pointer or highlighting effect to indicate the user's current gaze point. Finally, the frontend manages the selection timer, often referred to as "dwell time." If the system detects that a user has focused on a specific item for a predetermined duration, such as one second, a selection is triggered. This mechanism is crucial for preventing accidental activations, known as the "Midas Touch" problem, as the user's eyes move across the screen [9].

3.4.2 The Python Backend (The Inference Engine)(maybe not necessary)

The project plans to utilize a local Python-based backend to manage the specialized computational requirements of gaze estimation. Since most open-source eye-tracking libraries, including EyeGestures [6] and EyeTrax [7], are optimized for the Python ecosystem, this layer acts as the primary inference engine. The backend follows a systematic three-step cycle to ensure continuous data flow. It begins by capturing the raw video feed directly from the device's integrated webcam. These frames are then processed through the selected eye-tracking model to calculate the specific screen coordinates where the user is looking. Finally, the backend pushes these coordinates to the Flutter application in real-time, allowing the visual interface to react instantly to the user's eye movements.

3.4.3 Architectural Advantages

This proposed separation between the tracking logic and the display layer provides several strategic benefits. By offloading the mathematical heavy lifting to a background process, the system ensures higher reliability; even if the eye-tracking calculations encounter momentary lag, the Flutter user interface remains fluid and responsive without freezing. This approach also simplifies the development process, as it allows for the testing and swapping of different eye-tracking libraries within the Python environment without requiring a complete rewrite of the frontend code. Furthermore, this lightweight design is specifically intended to function on standard consumer webcams, directly supporting the project's core objective of creating a low-cost communication tool that does not require specialized infrared hardware.

3.5 Evaluation and Testing

To comprehensively validate the proposed decoupled AAC architecture, the system will undergo rigorous quantitative and qualitative evaluation. The testing methodology must differentiate between raw algorithmic tracking failures (backend) and interface design flaws (frontend).

3.5.1 Algorithmic Accuracy and Precision

To evaluate the fundamental suitability of the selected computer vision models, both accuracy and precision will be measured. Accuracy describes how close the estimated gaze position is to the intended target, while precision reflects how stable the gaze estimate remains (i.e., the amount of jitter) when the user fixates on a single point [9].

Evaluation will be performed using predefined ground-truth targets. A grid-based interface will be displayed on the Flutter or Cboard frontend, with target points appearing at random locations. Participants will be instructed to look directly at each target as it appears.

Accuracy will be quantified using angular error, computed as:

$$\theta = \arctan\left(\frac{d}{D}\right)$$

where:

- d is the Euclidean distance between the measured gaze point and the target location on the screen.
- D is the physical viewing distance between the user and the screen.

This results in an angular error, which is the standard metric in eye-tracking research [4, 9]. Precision will be assessed by computing the standard deviation of the gaze estimates while the user fixates on a single static target. Smaller variations indicate highly stable tracking, which is essential for interacting with specific bounding boxes on the UI.

3.5.2 Robustness to Head Movement and Environmental Variables

Because traditional fixed-head constraints are unacceptable for AAC users, experiments will be carried out under both fixed-head and free-head conditions. To test the efficacy of the Perspective-n-Point (PnP) algorithm [11] and MediaPipe Face Mesh [10], users will be instructed to perform controlled head movements across pitch, yaw, and roll axes. The system will be evaluated on "Landmark Stability," measuring whether the 3D facial tracking catastrophically fails or loses the pupil benchmark during extreme head rotation.

Furthermore, environmental robustness will be tested by introducing corrective eyewear. The evaluation will measure the model's F1-score in detecting eye geometry when the pupil is partially obscured by thick frames, refractive distortions, or specular glare from ambient room lighting.

3.5.3 System Latency and Architectural Performance

Given the split architecture of the proposed system, computational overhead must be evaluated. The Python inference engine relies heavily on real-time data streaming to the frontend via WebSockets. To prevent the visual cursor from desynchronizing with the user's actual eye movement, the system's end-to-end latency will be measured. The primary performance metric will be the system's ability to maintain a minimum processing speed of 30 frames per second (fps) on standard consumer hardware, ensuring fluid communication without freezing [5].

4 Deliverables

The deliverable for **Phase One** is the comprehensive Project Plan. It outlines the motivation, core concepts of webcam-based gaze estimation, and the architectural approach. This phase also includes the timeline, risk analysis, and hardware constraints for standard consumer devices, specifically addressing the limitations of relying on standard RGB cameras (like those on an iPad) for eye contrast and lighting, as opposed to the specialized infrared (IR) cameras used in commercial trackers.

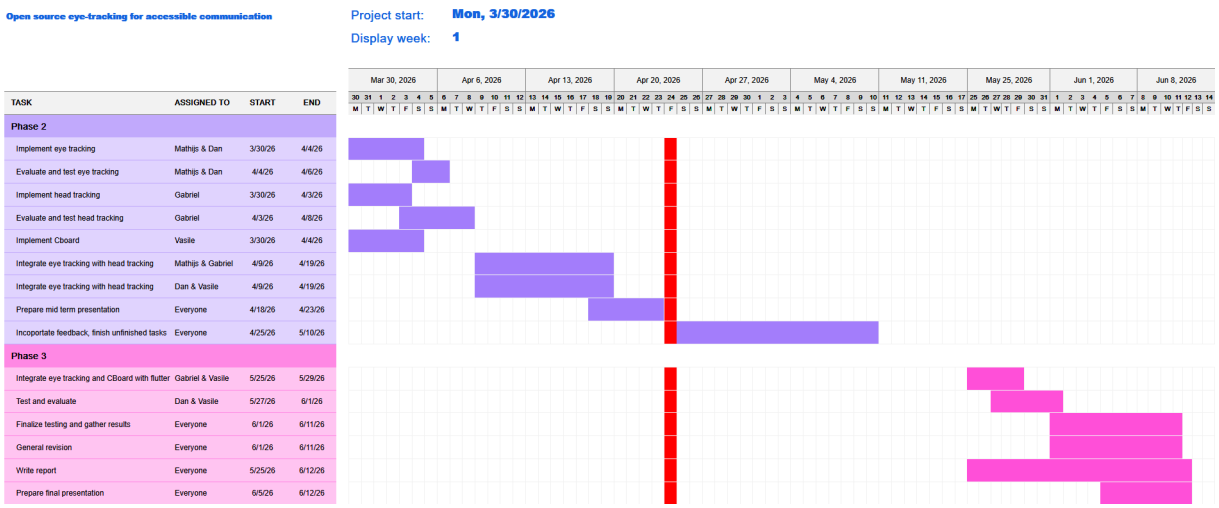
In **Phase Two**, the focus shifts to technical implementation. The deliverables include the configured inference backend, the integrated gaze-estimation algorithms (such as EyeGestures or EyeTrax), and a functional proof-of-concept for 3D head-pose compensation. A mid-project presentation and a layman's text describing the accessibility impact will also be provided.

Lastly, in **Phase Three**, the deliverables are determined by the outcomes of the previous phase. If the eye-tracking accuracy meets the required threshold for communication, the focus will be on the finalized Flutter application and UI integration. If further refinement is needed, the deliverable will consist of an optimized eye-tracking model with a comparative analysis of different algorithmic approaches. A final project report outlining the outcomes, evaluation metrics, and conclusions will accompany the final software.

Table 1: Project Deliverables

No.	Name	Description
D0	Phase One Project Plan	Comprehensive plan describing the problem, gaze-tracking approaches, and the system architecture.
D1	Backend Inference Engine	A Python-based backend capable of capturing webcam feeds and outputting raw gaze coordinates.
D2	PoC Eye-Tracking System	Implementation of 3D landmark detection and head-pose compensation logic.
D3	Phase Two Presentation	Presentation regarding the accuracy and stability of the tracking algorithms.
D4	Layman's Text	A non-technical explanation of the system's potential for AAC users.
D5	Optimized Tracker/App	Finalized eye tracker or a integrated Flutter app, depending on Phase Two performance.
D6	Evaluation	Comparative analysis of accuracy, precision, and robustness under different lighting/head-movement conditions.
D7	Final Project Report	Comprehensive documentation of the final state of the project, including software and results.

5 Time Management



6 Risk analysis

To ensure the successful development and deployment of the AAC system, several technical, environmental, and usability risks must be anticipated and mitigated.

6.1 Gaze Tracking Risks

There is a possibility that gaze tracking may not perform well on smaller devices. On small screens, the selectable elements may be too close together for the gaze tracker to reliably determine which item the user is looking at. As a result, the system might not be able to distinguish between nearby targets.

To address this possible limitation, the GazeTracker [14] library could be used as a fallback option. Instead of estimating a precise gaze point on the screen, this library detects the general direction in which the user is looking. This allows for an alternative interaction method based on gaze direction, where users can navigate by looking left, right, up or down to scroll through options. As a result, interaction remains possible without relying on precise gaze point estimation.

Additionally, head movement may significantly reduce eye-tracking accuracy. To address this, confidence levels in gaze estimation can be incorporated. When confidence drops below a defined threshold, the system can fall back to head movement compensation.

6.2 System Latency and Architectural Bottlenecks

The decoupled architecture introduces the risk of communication latency between the Python inference engine and the Flutter/Cboard frontend. If the WebSocket transmission of coordinates experiences lag, the visual feedback will desync from the user's actual eye movements. This delay can severely disrupt the dwell-time selection mechanism, leading to accidental selections (the "Midas Touch" problem) and user frustration. To mitigate this, the payload transferred between the backend and frontend will be restricted to lightweight JSON strings, and the Python processing loop will be optimized to maintain a minimum of 30 frames per second.

6.3 Project Scope and Implementation Complexity

A significant risk to the project timeline is the technical complexity of developing a full-scale, production-ready application. Integrating the custom Python inference engine with a fully functional Flutter frontend and subsequently linking it to the Cboard communication interface, requires substantial software engineering effort. There is a high probability that unforeseen integration challenges, UI/UX refinement, and cross-platform debugging could consume more time than allocated.

If completing the entire system proves infeasible withing the available time frame, the focus will shift towards improving and validating individual components. This approach enables more thorough testing and will provide a solid foundation for future development.

7 Overview of who did what

Table 2: Distribution of project responsibilities among group members.

Team Member	Deliverables
Mathijs (i6320858)	• **Eye Tracking** • **Time Management** • **Risk Analysis Section** • Secondary: Refined and checked the structure of the proposal.
Dan (i6320800)	• **Head Movement and Pose Compensation** • **Risk Analysis Section** • **State of the art section** • Secondary: Refined and checked the structure of the proposal.
Gabriel (i6321439)	• **Application Architecture** • **Deliverables Section** • Secondary: Refined and checked the structure of the proposal.
Vasile (i6321735)	• **The Communication Frontend: Cboard** • **Context and Motivation Section** • Secondary: Refined and checked the structure of the proposal.

References

[1] D. R. Beukelman and J. C. Light, *Augmentative and Alternative Communication: Supporting Children and Adults with Complex Communication Needs*, 5th ed. Paul H. Brookes Publishing Co., 2020, excerpt accessed online. [Online]. Available: https://brookespublishing.com/wp-content/uploads/2020/05/BeukelmanExcerpt0506-1.pdf

[2] Y. Elsahar, S. Hu, K. Bouazza-Marouf, D. Kerr, and A. Mansor, "Augmentative and alternative communication (aac) advances: A review of configurations for individuals with a speech disability," *Sensors*, vol. 19, no. 8, p. 1911, 2019.

[3] K. Krafka, A. Khosla, P. Kellnhofer, H. Kannan, S. Bhandarkar, W. Matusik, and A. Torralba, "Eye tracking for everyone," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2176–2184.

[4] E. Guestrin and M. Eizenman, "General theory of remote gaze estimation using the pupil center and corneal reflections," *IEEE Transactions on Biomedical Engineering*, vol. 53, no. 6, pp. 1124–1133, 2006.

[5] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. MacGlashan, F. Bogo *et al.*, "Mediapipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019.

[6] NativeSensors, "Eyegestures: Open source eye-tracking software," https://github.com/NativeSensors/EyeGestures, 2025, accessed: 2026-03-22.

[7] C. Zhang, "Eyetrax: Webcam-based eye tracking library," https://github.com/ck-zhang/EyeTrax, 2025, accessed: 2026-03-22.

[8] M. Lotfi, "Owleye: Transform your webcam into an eye tracker," https://github.com/MustafaLotfi/Owleye, 2021, accessed: 2026-03-22.

[9] K. Holmqvist, M. Nyström, R. Andersson, R. Dewhurst, H. Jarodzka, and J. Van de Weijer, *Eye tracking: A comprehensive guide to methods and measures.* OUP Oxford, 2011.

[10] Google, "Mediapipe face mesh and iris tracking," https://developers.google.com/mediapipe/solutions/vision/face_landmarker, 2024, accessed: 2026-03-22.

[11] V. Lepetit, F. Moreno-Noguer, and P. Fua, "Epnp: An accurate o(n) solution to the pnp problem," *International journal of computer vision*, vol. 81, no. 2, pp. 155–166, 2009.

[12] C. Foundation, "Cboard: An open-source aac web application," https://www.cboard.io/, 2024, accessed: 2026-03-22.

[13] Google, "Flutter: Multi-platform ui toolkit," https://flutter.dev/, 2024, accessed: 2026-03-22.

[14] A. Lamé, "Gazetracking: Webcam-based eye tracking library," https://github.com/antoinelame/GazeTracking, 2024, accessed: 2026-03-22.